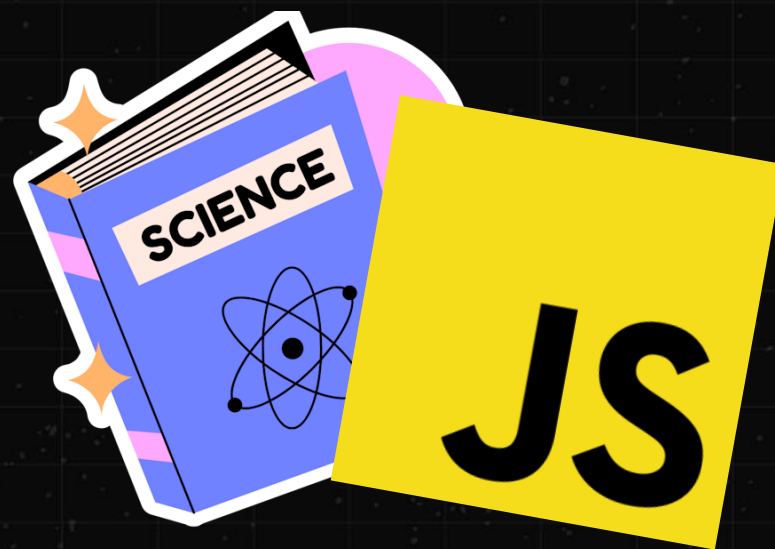


ЛЕКЦИЯ #6

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Будем говорить про массивы в общем и массивы в JavaScript
- * Особенность массивов в том, что они лежат в основе большинства других структур данных
- * Например, хэш-таблица или вектор под капотом используют массив
- * Кроме этого, сегодня мы разберем понятие алгоритмической сложности

ПАМЯТЬ – ЭТО ПРОСТО БОЛЬШОЙ МАССИВ

- * Процессор может **считать последовательность байт** начиная с заданного адреса и загрузить его в свои регистры
- * Мы либо **заранее знаем**, сколько байт нужно считать для работы с данными
- * Например, размер Uint32 заранее известен и равен 4 байта
- * Либо мы **храним размер наших данных в самой памяти рядом с данными**
- * Тогда мы можем **предварительно прочитать размер**, а дальше уже считать сами данные

НАПРИМЕР



Хотим прочитать байты числа в формате Uint32: считываем первый байт по его адресу (индексу) в памяти и еще следующие за ним 3 байта

А КТО ОПРЕДЕЛЯЕТ
ПО КАКОМУ АДРЕСУ
ХРАНЯТСЯ
ДАННЫЕ?



ТУТ ДВА ВАРИАНТА

- * Либо сам программист – но это очень «шаткая» логика и **таким занимаются при программировании на очень низком уровне**
- * В остальных случаях это **задача компилятора/среды и аллокатора памяти**
- * У нас будет про это лекция

ПРОСТЫЕ ТИПЫ

- * Высокоуровневые языки программирования **вводят множество стандартных типов**
- * Например, числа или логические значения, пустое множество
- * Размер этих типов заранее известен и **мы работаем с ними как с единым целым**
- * Например, сложение двух чисел **создает новое число**
- * К сожалению, для создания сложных программ этого не достаточно
- * Например, мы хотим работать **не с одним числом, а со списком чисел**

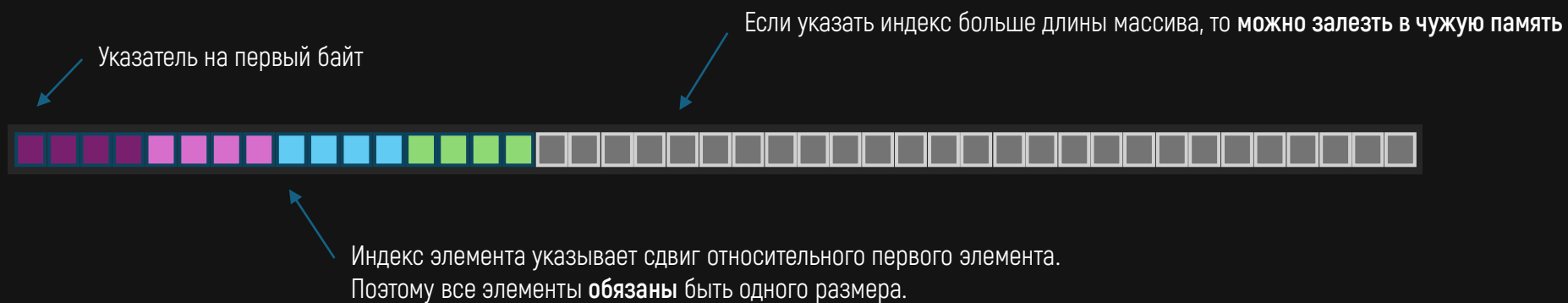
НАМ НУЖЕН
МАССИВ
ЧИСЕЛ?



В ТОЧКУ

- * Массив – это **одна из важнейших структур данных** в программировании вообще
- * При этом её реализация до смешного тривиальна и она в общем случае **не требует никакой дополнительной информации в рантайме**
- * Абстракции, которые **не имеют цены в рантайме** называют абстракциями с нулевой стоимостью
- * Все дело в том, что сама память это уже массив – и **можно просто «выделить массив поменьше»**

НАПРИМЕР



В ЧЕМ ТУТ СОЛЬ

- * Массив – это просто **некоторая непрерывная область памяти** логически побитая на ячейки одного заранее известного размера
- * Мы знаем адрес первого байта и знаем размер одного элемента – значит прочитать любой элемент массива это **простая арифметика над адресом**
- * Берем адрес первого байта и прибавляем к нему **индекс нужного нам элемента умноженный на размер самого элемента**
- * Фактически, индекс определяет **смещение относительно первого элемента массива**

ДОСТУП К ЭЛЕМЕНТУ МАССИВА – КОНСТАНТА

- * Нам не важно сколько элементов в массиве –
доступ к любому из них будет «стоять» одинаково
- * Это означает, что **нет зависимости между количеством элементов и временем**
доступа к любому элементу по его индексу
- * Но мы уже знаем, что кэш-память процессора вносит свои коррективы

У ЭЛЕМЕНТОВ В МАССИВЕ ЕСТЬ ПОРЯДОК

- * Это позволяет обойти элементы массива ровно так, **как они хранятся в памяти**
- * А также позволяет **менять элементы местами**, для создания конкретного порядка
- * Этот процесс **называют сортировкой**
- * Алгоритмов сортировки массива **существует великое множество**
- * **У каждого есть свои плюсы и минусы** – идеального нет
- * Мы не будем изучать эти алгоритмы на курсе
- * Я рекомендую почитать [Р. Лафоре](#)

МАССИВЫ И КЭШ

- * Элементы массивов хранятся в памяти «рядышком» –
это хорошо для использования кэша процессора
- * Но мы помним, что эффективность кэша определяется не только этим,
а в каком порядке мы читаем данные массива
- * Если мы будем «прыгать» по массиву, то **эффективность будет хуже**

ЧТО БУДЕТ, ЕСЛИ
СЧИТАТЬ ИНДЕКС,
КОТОРОГО НЕТ?



А ВОТ ТУТ ИНТЕРЕСНО

- * В высокоуровневых ЯП мы получим либо исключение, либо специальное значение, означающее «пустоту» (в JS это `undefined`)
- * А в низкоуровневом ЯП мы можем считать часть памяти, которая уже не является массивом – и **это может привести к неизвестным последствиям**
- * Такое поведение называют **Undefined Behavior** или просто **UB**

UNDEFINED BEHAVIOR



В ЧЕМ СМЫСЛ UB

- * Поведение программы с UB **нельзя предсказать на 100%**
- * Программа может считать чужую память и упасть с ошибкой
- * Или **продолжить работать и это приведет к каким то последствиям**
- * На разных системах, с разными компиляторами или средами исполнения **результат может отличаться**
- * **Главное свойство безопасных ЯП – это отсутствие UB**



```
#include <stdio.h>
```

```
int main() {
```

```
    // Максимальное значение 32-битного int
```

```
    int a = 2147483647;
```

```
    // Компилятор C предполагает, что переполнений в программе быть не должно,
```

```
    // поэтому он может просто удалить этот код, что изменит логику работы
```

```
    if (a + 1 < a) {
```

```
        printf("Меньше\n");
```

```
    }
```

```
    return 0;
```

```
}
```


РЕФЛЕКСИРУЕМ

- * UB может появиться не только из-за того, что вы написали какой-то плохой код
- * Компилятор может с оптимизировать код, полагаясь на то, что **в нем нет не может произойти ничего плохого**
- * А в реальности это **изменит логику нашей программы**
- * В C проверка многих базовых контрактов – **обязанность программиста**
- * В JS (в идеале) UB произойти не может

ПОЧЕМУ В ИДЕАЛЕ?



JS ИСПОЛНЯЕТСЯ ВИРТУАЛЬНОЙ МАШИНОЙ

- * А она уже с высокой долей вероятностью написана на C или C++, который UB допускает
- * Безобидный JS код из-за ошибки в VM может привести к UB
- * На практике такое происходит регулярно...

ПОЧЕМУ НЕ
ИСКЛЮЧИТЬ
UV ВЕЗДЕ?



ДВА СТУЛА

- ✱ Низкоуровневые ЯП позволяют писать очень эффективный код
- ✱ Высокоуровневые ЯП дают безопасность, но у неё есть цена
- ✱ Долгое время считалось, что разумный компромисс не возможен
- ✱ Языки типа Rust могут гарантировать отсутствие UB за счет строгих правил языка и компилятора
- ✱ Ценою будет усложнение самого ЯП, но и тут есть нюансы...

ДОСТУП К НЕИНИЦИАЛИЗИРОВАННОЙ ПАМЯТИ

- * В С очень важно **перед тем, как считать память**, быть уверенным, что там что-то есть
- * Если **создать переменную без значения или создать массив и не заполнить его** – то можно прочесть «мусорную» память и получить UB
- * Чтение неинициализированной памяти, наряду, с выходом за пределы массива – **источник самых страшных ошибок в ПО**



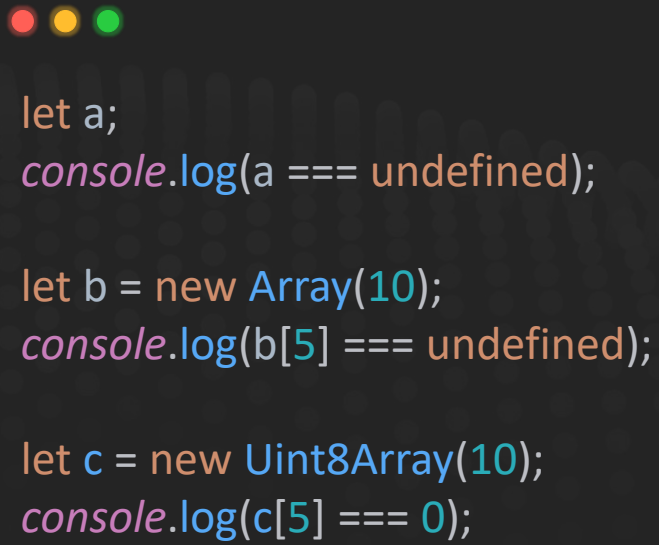
```
#include <stdio.h>

int f() {
    int i = 13;
    return i;
}

int g() {
    int i;
    return i;
}

int main() {
    f();
    printf("Результат вызова g(): %d\n", g()); // Что здесь будет?
}
```

JS HAC CTPAXYET



```
let a;  
console.log(a === undefined);  
  
let b = new Array(10);  
console.log(b[5] === undefined);  
  
let c = new Uint8Array(10);  
console.log(c[5] === 0);
```


РЕФЛЕКСИРУЕМ

- * JS гарантирует **отсутствие возможности прочитать неинициализированную память**
- * Таким образом он **защитил нас от UB, но это не бесплатно**
- * Любой безопасный ЯП делает либо так,
либо **требуется явное задания значения по умолчанию**
- * Rust **просто не скомпилирует такую программу (без unsafe)**
- * В C и похожих ЯП – **за этим следит сам программист**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОСТАРАЙТЕСЬ ПОНЯТЬ СУТЬ

- * Чем ниже уровень абстракции, тем больше контрактов **должен проверять сам программист**
- * За счет этого программа может получиться максимально эффективной, **но цена ошибки фатальна**
- * Rust **стремиться к безопасности без цены в рантайме**, но и там есть нюансы

ВЕРНЕМСЯ К МАССИВАМ

- * По определению, массив должен иметь фиксированную «длину», а **размер у всех элементов должен быть одинаковый**
- * Это приводит к тому, что «настоящий» массив не может динамически «расти» или содержать элементы разных типов (массивы гомогенны)

ПОЧЕМУ
«НАСТОЯЩИЙ»
МАССИВ НЕ МОЖЕТ
РАСТИ?



НАПРИМЕР



РЕФЛЕКСИРУЕМ

- * В нашей **программе много чего**, а не только наш массив
- * И мы не можем гарантировать, **что после него в памяти не будет другого значения**
- * А значит и **не можем безопасно «расширить» массив**
- * Простейший способ – **выделить память большего размера**
и скопировать значения из старого туда
- * Мы обсудим это на отдельной лекции

HO B JS JE
HE TAK?



В JS МАССИВЫ – НЕ МАССИВЫ

- * Если смотреть на них с точки зрения фундаментальных принципов
- * Это куда **более мощественная структура данных**, которая использует «настоящие» массивы под капотом
- * Гетерогенность массивов в JS достигается за счет того, что все элементы в нем **хранятся в виде указателей** (некоторые таковыми не являются)
- * **Более близки к «настоящим» массивам** в JS
 - это типизированные массивы (но и там есть улучшения)

МАССИВЫ В JS ПОДДЕРЖИВАЮТ «ДЫРКИ»



```
let a = [];
```

```
// Убийца производительности
```

```
a[100] = 42;
```

```
// Сразу просим нужный размер и заполняем его значением по умолчанию
```

```
let b = new Array(101).fill(0);
```

```
// Теперь все хорошо
```

```
b[100] = 42;
```

В ЧЕМ ПРОБЛЕМА

- * Создание дырок в массивах вынуждает VM использовать **другую структуру данных под капотом**
- * Это приводит к резкому падению производительности, **поэтому никогда так не делайте!**
- * Когда вы заранее выделяете длину и в дополнение инициализируете память «правильным» типом, **то VM и JIT скажут вам спасибо**

node --allow-natives-syntax

```
const a = new Array(100);
%DebugPrint(a); // HOLEY_SMI_ELEMENTS

for (let i = 0; i < a.length; i++) {
  a[i] = 0;
}

%DebugPrint(a); // Обратите внимание, все еще HOLEY_SMI_ELEMENTS

const b = [0, 1, 2, 3, 4, 5];
%DebugPrint(b); // PACKED_SMI_ELEMENTS

b[1000] = 42;
%DebugPrint(b); // Обратите внимание, теперь HOLEY_SMI_ELEMENTS

const c = new Array(100).fill(0);
%DebugPrint(c); // PACKED_SMI_ELEMENTS

const d = new Array(100).fill([]);
%DebugPrint(d); // PACKED_ELEMENTS
```


ОБРАТИТЕ ВНИМАНИЕ

- * В V8 у массивов есть свои «подтипы»: массивы с дырками (holey), упакованный (packed), а также по типам элементов
- * При этом тип определяется при создании массива и **может измениться в «худшую» сторону** (более абстрактный тип)
- * А вот в «лучшую» не может (стать более конкретным)
- * Иногда это может **быть причиной падения производительности**
- * Но не спешите везде писать **fill**!

КАК ДУМАЕТЕ, КТО БЫСТРЕЕ?

```
function mapWithNewArray(arr, mapFn) {  
  const result = new Array(arr.length);  
  
  for (let i = 0; i < arr.length; i++) {  
    result[i] = mapFn(arr[i]);  
  }  
  
  return result;  
}
```

```
function mapWithFill(arr, mapFn) {  
  const result = new Array(arr.length).fill(0);  
  
  for (let i = 0; i < arr.length; i++) {  
    result[i] = mapFn(arr[i]);  
  }  
  
  return result;  
}
```

```
function mapWithPush(arr, mapFn) {  
  const result = [];  
  
  for (let i = 0; i < arr.length; i++) {  
    result.push(mapFn(arr[i]));  
  }  
  
  return result;  
}
```



--- Размер массива: small (100 элементов) ---

new Array : 0.010 ms (среднее за 10 итераций)

fill + assign : 0.003 ms (среднее за 10 итераций)

push : 0.004 ms (среднее за 10 итераций)

--- Размер массива: medium (10 000 элементов) ---

new Array : 0.034 ms (среднее за 10 итераций)

fill + assign : 0.252 ms (среднее за 10 итераций)

push : 0.090 ms (среднее за 10 итераций)

--- Размер массива: large (1 000 000 элементов) ---

new Array : 1.805 ms (среднее за 10 итераций)

fill + assign : 2.040 ms (среднее за 10 итераций)

push : 6.017 ms (среднее за 10 итераций)

--- Размер массива: huge (10 000 000 элементов) ---

new Array : 20.029 ms (среднее за 10 итераций)

fill + assign : 22.940 ms (среднее за 10 итераций)

push : 80.872 ms (среднее за 10 итераций)

РЕФЛЕКСИРУЕМ

- * Нет никакого смысла использовать `fill`, если вы потом все равно заполните массив
- * Напротив, мы даже видим ухудшение из-за лишней работы
- * А вот предварительное задание длины значительно ускоряет работу по сравнению с динамическим ростом через `push`
- * Причины этого мы обсудим на отдельной лекции

НО СТОИТ НЕМНОГО ИЗМЕНИТЬ...



```
function badMap(arr, mapFn) {  
  const result = []; // Вместо new Array(arr.length)  
  
  // Обход с конца провоцирует дырки  
  for (let i = arr.length; i--;) {  
    result[i] = mapFn(arr[i]);  
  }  
  
  return result;  
}
```

ПРОИЗВОДИТЕЛЬНОСТЬ ПАДАЕТ В 32 РАЗА!

- * И в 10 раз хуже варианта с `push`!
- * При этом если вернуть `new Array(arr.length)`, то производительность снова становится самой лучшей (даже с обратным порядком)
- * Делайте выводы

РЕЗЮМИРУЕМ

- * Массивом с дырками или разряженным массивом называется такой массив, где между элементами в памяти может быть «пустота»
- * Такой массив не может быть представлен через «настоящий» массив и движок вынужден использовать другую структуру данных
- * Предварительное задание длины массива снимает негативный эффект от дырок
- * Если вы сами не инициализируете элементы такого массива, то используйте `fill` – VM скажет вам спасибо

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

ДА ВСЕ ПРОСТО

- * Задавайте длину массива, **если она вам известна**
- * **Никогда не создавайте массивы с дырками**
- * Старайтесь **не смешивать разные типы данных** в одном массиве

ОГРАНИЧЕНИЕ МАССИВОВ В JS

- * По спецификации массивы в JS не могут содержать более $2^{32} - 1$ элементов
- * Но в реальности, лучше не превышать $2^{31} - 1$ элементов
- * Вы можете создать массив с индексами и больше $2^{32} - 1$, но такие элементы не будут обрабатываться как элементы массива



```
let a = new Array(2 ** 32); // RangeError: Invalid array length

let b = [];
b[2 ** 32] = 42; // Так можно, но это значение уже будет добавлено "не в массив"
console.log(b.length === 0);
```

ДЕЛО В РАЗРЯДНОСТИ

- * Спецификация JS сделана с оглядкой на 32 битную архитектуру – иначе код мог не работать в 32 битных виртуальных машинах
- * Даже $2^{32} - 1$ на 32 битной программе будет проблемой – ведь нам потребуется вся доступная память и это приведет к проблемам
- * С другой стороны, даже без этого ограничения мы связаны с $2^{53} - 1$ – после этого числа в `number` теряется точность целого числа

OUT OF MEMORY

- * Даже если вы аллоцировали массив нужной длины – **это еще не победа**
- * Например, `new Array(2 ** 32 - 1)` не выделяет физическую память
- * Память под массив выделяется «виртуальная» и по мере наполнения реальными данными **она может физически закончиться**
- * А `new Array(2 ** 32 - 1).fill(0)` – выделяет
- * Программа **завершится с ошибкой Out Of Memory (OOM)**
- * У нас будет про это лекция

А ТИПИЗИРОВАННЫЕ МАССИВЫ?



ТУТ НЮАНСЫ

- * Ограничение для 32-битной системы никуда не делось и на практике мы получим ошибку при значении больше $2^{**} 31 - 1$
- * В 64 битной Node.js можно создать типизированный массив больше
- * Но мы все равно ограничены доступной памятью системы и памятью, к которой имеет доступ виртуальная машина
- * Мы помним, что даже 64 битные процессоры не могут адресовать $2^{**} 64 - 1$ – в реале значение куда меньше

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

НЕ ВСЕ ТАК СТРАШНО

- * Даже $2^{31} - 1$ – это огромное значение (больше 2 миллиардов)
- * 64 битные системы могут адресовать куда больше, но **если памяти физически нет**, то никуда не денешься
- * В случае с JS еще **специфика из-за виртуальной машины** и особенностей числовых типов
- * С **типизированным массивами** будем разбираться на следующей лекции

ОПЕРАЦИИ С МАССИВАМИ

- * Массив – это структура данных, которая реализует **упорядоченный список элементов** (включая другие массивы)
- * Мы знаем, что **каждому элементу соответствует свой индекс**
- * **Чтение элемента по индексу** – константная операция
- * Но что это значит и **что еще мы можем делать с массивами?**

ПОНЯТИЕ АЛГОРИТМИЧЕСКОЙ СЛОЖНОСТИ

- * Нам нужно как-то **оценивать эффективность работы** применяемых алгоритмов
- * Можно конечно просто гонять бенчмарки, но **хочется чего то более предсказуемого**
- * Поэтому придумали **способ асимптатической оценки сложности**
- * В нем оценивают примерную зависимость между количеством входных данных и **временем работы алгоритма или потребления им памяти**
- * Сложность принято записывать как **O -функцию**, например, $O(n)$

$O(1)$ – КОНСТАНТНАЯ СЛОЖНОСТЬ

- * Означает, что зависимости между количеством данных и временем или памятью нет
- * Например, **чтение или запись элемента массива по индексу** всегда константа по времени и памяти
- * Алгоритм «быстрой» сортировки не создает дополнительный массив в памяти, значит **у него константная сложность по памяти**

$O(n)$ – ЛИНЕЙНАЯ СЛОЖНОСТЬ

- * Означает, что **алгоритм линейно связан по времени или памяти** с количеством входных данных
- * Например, чтобы найти некоторый элемент массива по его значению, **в худшем случае придется обойти все его элементы**
- * Значит сложность по времени у такого алгоритма будет $O(n)$

ИМЕННО **худший**
случай?



НЕ ВСЕГДА

- * Некоторые алгоритмы **имеют стабильную сложность в любом случае**
- * В остальных случаях оценивают **среднюю сложность и сложность в худшем случае**
- * Например, «быстрая» сортировка имеет (по времени) **среднюю сложность $O(n \log n)$, а худшую $O(n^2)$**

ОТБРАСЫВАЕМ КОЭФФИЦИЕНТЫ

- * Если ваш алгоритм, например, делает два прохода по всем элементам, то **его сложность все равно $O(n)$**
- * Смысл в том, что мы **не показывает «конкретную» сложность алгоритма**, а примерную – этого вполне достаточно для анализа
- * С точки зрения математики, функция $O(g(x))$ – это предел

$$O(g(x)) = \left(\frac{f(x)}{g(x)} \right) = \text{const} > 0 = \lim_{x \rightarrow \infty}$$

$O(n^2)$ – КВАДРАТИЧНАЯ СЛОЖНОСТЬ

- * Это уже маркер «тяжелого алгоритма», например, для 100 элементного массива такой алгоритм будет вынужден выполнить 10 000 «операций»
- * Или **выделить память** под столько элементов
- * Многие алгоритмы сортировки в худшем случае скатываются к $O(n^2)$

$O(n^x)$ – ПОЛИНОМИНАЛЬНАЯ СЛОЖНОСТЬ

- * Смысл такой же, как и с квадратом, только **функцию** растет куда быстрее
- * Нужно избегать алгоритмов с такой сложностью

$O(\log n)$ – ЛОГАРИФМИЧЕСКАЯ СЛОЖНОСТЬ

- * При такой сложности **зависимость описывается функцией логарифма**
- * Основание логарифма нам здесь не важно
- * Это пример «хорошей» сложности, например, бинарный поиск в отсортированном массиве имеет **стабильную логарифмическую сложность по времени**
- * Для $2^{32} - 1$ элементов будет сделано всего 32 операции сравнения

$O(n \log n)$ – ЛИНЕЙНО-ЛОГАРИФМИЧЕСКАЯ

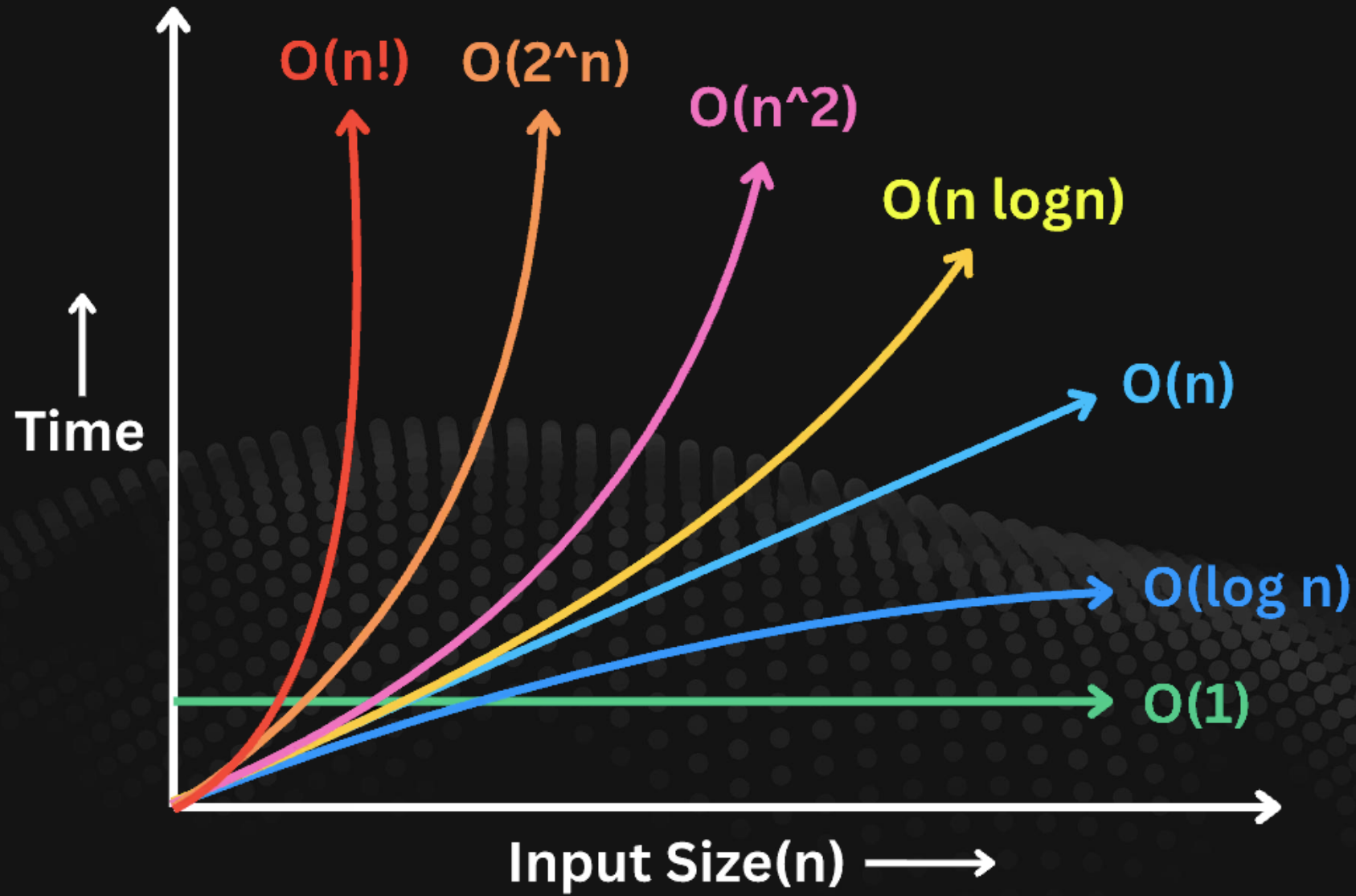
- * Хуже линейной, но куда лучше квадратичной
- * Многие алгоритмы сортировки в среднем имеют такую сложность по времени

$O(n!)$ – ФАКТОРИАЛ

- * Очень «плохая» сложность – функция факториала растет очень быстро
- * $10! = 3\,628\,800$
- * Например, алгоритм **генерации перестановок элементов массива** имеет такую сложность

ЕСТЬ И ДРУГИЕ СЛОЖНОСТИ

- ✱ Как видите, мы просто берем за основу одну из математических функций
- ✱ Чтобы лучше понимать разницу между этими сложностями – можно держать в голове графики этих функций



ЧТО НАМ
ЭТО **ДАЛО?**



МЫ МОЖЕМ АНАЛИЗИРОВАТЬ ЭФФЕКТИВНОСТЬ

- * Применяемых алгоритмов без необходимости бенчмарков – это особенно **важно на этапе анализа и выбора алгоритма**
- * Легко держать в голове, что например **логарифм лучше линейной сложности**
- * Но реальный мир вводит свои коррективы...

КАКИЕ ТУТ НЮАНСЫ

- * Два алгоритма с одной сложностью **скорее всего** будут работать за разное «реальное» время
- * Причины тут как раз в «отбрасываемых коэффициентах», использовании кэша алгоритмом или других нюансов
- * А бывает, что алгоритм с худшей сложностью на практике работает лучше, например, из-за того же кэша
- * Так что **бенчмарки все равно нужны**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

ВСЕ НЕ ТАК СТРАШНО

- * С опытом вы будете заранее понимать, где стоит, а где нет – учитывать сложность алгоритма
- * Например, для небольших массивов линейная сложность – это «хорошая» сложность для любых операций
- * За счет кэша все будет работать «моментально»

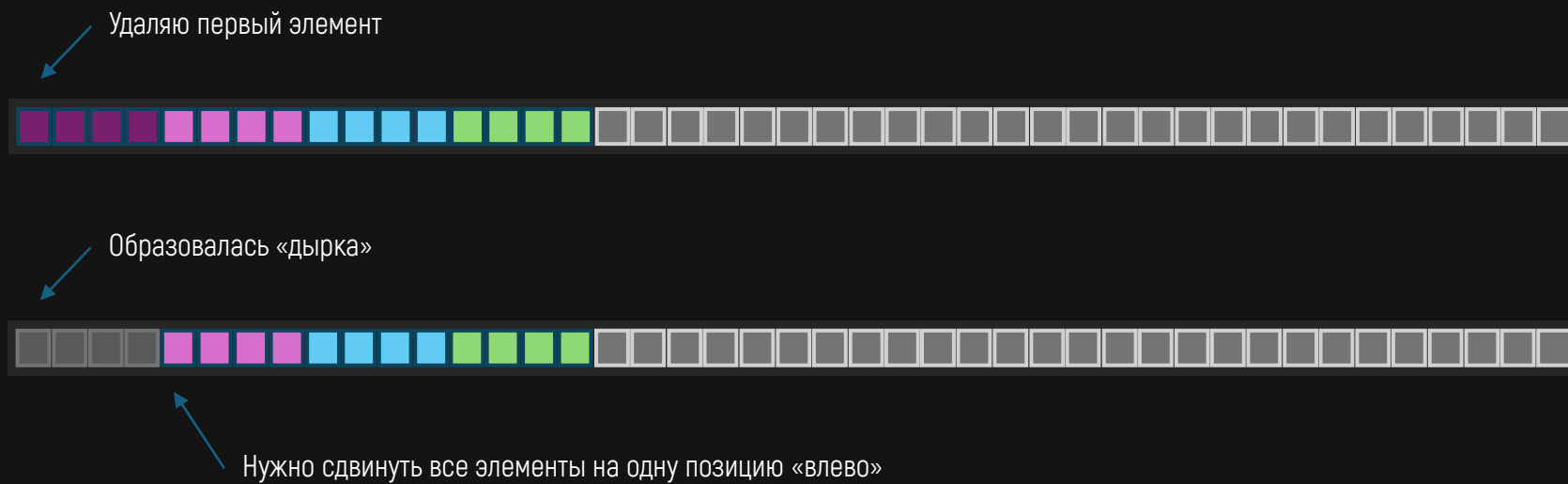
ВСТАВКА И УДАЛЕНИЕ ИЗ МАССИВА

- * Вставка в конец массив (при условии, что длина позволяет) – это стабильно константная операция
- * Удаления с конца тоже всегда константа
- * А вот вставка и удаление из произвольного места – в худшем случае это $O(n)$

ПОЧЕМУ $O(n)$?



НАПРИМЕР, УДАЛЕНИЕ

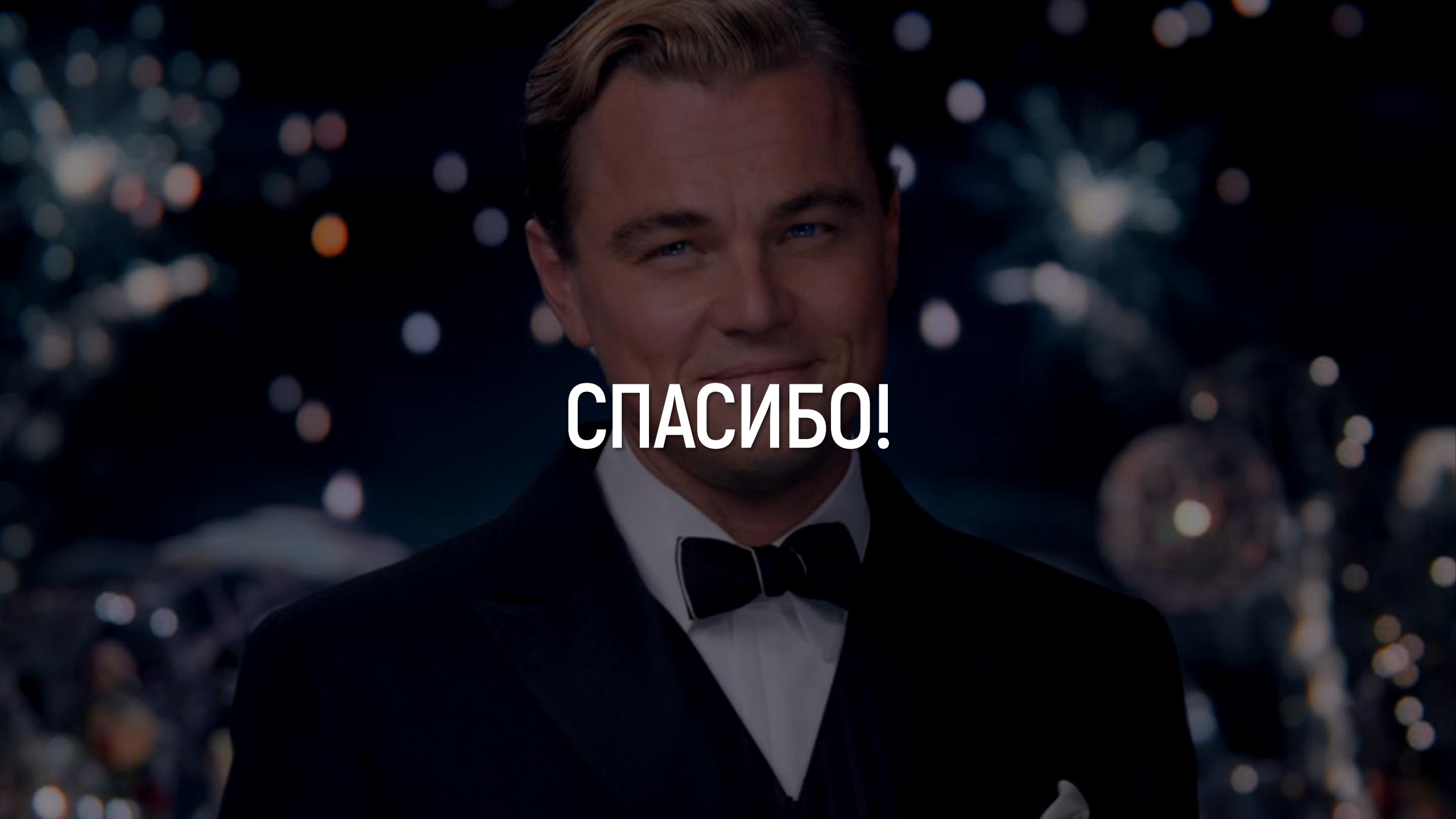


ДЛЯ НЕБОЛЬШИХ МАССИВОВ ЭТО НЕ КРИТИЧНО

- * Благодаря кэшированию операция все равно будет очень быстрой
- * Но на больших массивах разница будет огромной
- * В ДЗ к этой лекции вы сравните `push/pop` и `unshift/shift` в JS
- * Для таких задач массивы не подходят и применяются **другие структуры данных оптимизированные для этих операций**
- * Иронично, но большинство из них все равно основаны на массивах

ПОДВЕДЕМ ИТОГИ

- * Массив – это, вероятно, **самая важная** из существующих структур данных
- * Массив представляет собой **фиксированную непрерывную область памяти** логически «разделенную» на элементы одного размера
- * Элементы в памяти хранятся «друз за другом», благодаря чему можно эффективно использовать кэш процессора
- * Многие **более сложные структуры данных** «ПОД КАПОТОМ» основываются на массивах
- * Массивы в JS – **один из таких примеров**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!